色キューブ仕分けタスクにおける ACT 方策への SAE 介入

Physical AI 応用編1 最終課題　説明資料

1. 概要と目的

1.1 背景: 学習済みの動作生成モデルの推論動作を調整したい

ロボットの動作生成では、学習済みモデルの推論動作を少しだけ調整したい場面がある。pick and place であれば、物体を置く位置を学習データより前後左右にずらしたり、運搬中の軌道を高く／低くしたりといった調整である。

動作と言語を一緒に学習した基盤モデル（VLA など）であれば、言語指示で対応できることもある。しかし、直したい動作の概念を基盤モデルが学習できていない場合や、動作データだけで学習した特化モデルのように言語でラベル付けされていない場合には、条件を少し変えるだけでも、その条件でデータを取り直して追加学習（fine-tuning など）するのが一般的である。

そこで、モデルを再学習せず、**重みは固定したまま推論時に内部の活性へ介入して動作を変える**（activation steering）という方法が考えられる。この種の手法は LLM の分野で多く提案されており、その代表例が Sparse Autoencoder（SAE）と DiffMean である。

1.2 SAE と DiffMean

SAE（Sparse Autoencoder）は、モデルの中間層の活性ベクトルを、それより次元の大きい「特徴」の空間に写し、少数の特徴だけが働く疎な表現に分解する補助モデルである。元のモデルとは別に、元のモデルの活性を再構成するように学習する。個々の特徴は「赤色」「右へ動く」のように人が解釈できる概念に対応することが多く、ある出力に対応する特徴を見つけ（probing）、その活性を推論中に強めたり弱めたりして元のモデルに戻すことで、出力を変えられる（steering）。介入するのは活性であり、元のモデルの重みは変えない。LLM では、Anthropic の研究で Claude の中間層から「ゴールデンゲートブリッジ」に対応する特徴が見つかり、この特徴を強めると何を聞いてもゴールデンゲートブリッジの話をするようになったことが示されている [Templeton et al., 2024]。本課題では、入力ごとに活性の大きい上位 k 個の特徴だけを残して疎さを直接決める TopK SAE [Gao et al., 2024] を使った。

式で書くと、TopK SAE は中間層の活性 $\mathbf{h} \in \mathbb{R}^d$（本課題では $d = 256$）を次のように符号化・復号する。

$$\mathbf{f} = \mathrm{TopK}_k(W_{\mathrm{enc}}\,(\mathbf{h} - \mathbf{b}_{\mathrm{pre}}) + \mathbf{b}_{\mathrm{enc}}) \in \mathbb{R}^m, \qquad \hat{\mathbf{h}} = W_{\mathrm{dec}}\,\mathbf{f} + \mathbf{b}_{\mathrm{dec}} = \sum_i f_i\,\mathbf{d}_i + \mathbf{b}_{\mathrm{dec}}$$

ここで m は辞書の大きさ（本課題では 2048）、TopK_k は値の大きい上位 k 個だけを残して（負の値は 0 にして）他を 0 にする操作、$\mathbf{d}_i$ は W_{dec} の第 i 列（特徴 i の方向を表す単位ベクトル）である。つまり活性 $\mathbf{h}$ は、k 個の特徴方向 $\mathbf{d}_i$ を強さ f_i で足し合わせたものとして近似される。学習では、再構成誤差 $\|\mathbf{h} - \hat{\mathbf{h}}\|_2^2$ を最小化する（ほかに、使われなくなった特徴を復活させる補助損失を小さい重みで加えた）。疎さは k で直接決まるので、L1 正則化の係数を調整する必要はない。

steering では、特徴の活性 $\mathbf{f}$ を書き換えて $\mathbf{f}'$ とし（例: 特徴 i を強める $f'_i = f_i + \alpha$、特徴 j を消す $f'_j = 0$）、その差だけを元の活性に足し戻す。

$$\mathbf{h}' = \mathbf{h} + W_{\text{dec}}\left(\mathbf{f}' - \mathbf{f}\right)$$

$\hat{\mathbf{h}}$ で $\mathbf{h}$ を丸ごと置き換えるのではなく差分だけを足すので、SAE が再構成できない成分 $\mathbf{h} - \hat{\mathbf{h}}$ はそのまま残り、介入しない部分の動作は壊れない。

DiffMean（平均差ベクトル）は、ある概念が「ある」入力と「ない」入力（例えば赤いキューブのときと青いキューブのとき）で中間層の活性の平均を取り、その差のベクトルを推論中の活性に足す方法である [Marks & Tegmark, 2023]。式で書くと、概念がある入力の集合を $\mathcal{A}$、ない入力の集合を $\mathcal{B}$ として、

$$\mathbf{v} = \frac{1}{|\mathcal{A}|}\sum_{\mathbf{h}\in\mathcal{A}}\mathbf{h} \;-\; \frac{1}{|\mathcal{B}|}\sum_{\mathbf{h}\in\mathcal{B}}\mathbf{h}, \qquad \mathbf{h}' = \mathbf{h} + \alpha\,\mathbf{v}$$

である（α は介入の強さ）。例えば赤を青に変えたいときは、$\mathcal{A}$ を青いキューブのときの活性、$\mathcal{B}$ を赤いキューブのときの活性にとる。SAE が「どの特徴を動かすか」を辞書から選ぶのに対し、DiffMean は2群の差という1本の方向だけを使う。学習が不要で、2群のデータだけで作れる。LLM の steering のベンチマークである AxBench では、DiffMean が SAE より高い steering 性能を示した [Wu et al., 2025]。そこで本課題では、DiffMean を SAE と比べるベースラインとして使った。

1.3 probing と steering のギャップ

ある出力に関わる特徴を見つけられても（probing できても）、その特徴を強めたときに狙った要素だけが変わるとは限らない。疎な特徴の空間に分解しても、1つの概念の情報が複数の特徴に少しずつ分散したり、他の概念と同じ特徴に混ざったりするため、1つの特徴を動かすと狙った方向からずれた変化（副作用）が出る。このずれを減らすために、疎さを直接制御して再構成と疎さのバランスを改善する TopK SAE [Gao et al., 2024] や、小さい辞書（粗い特徴）から大きい辞書（細かい特徴）までを入れ子にして同時に学習し、概念の分離を改善する Matryoshka SAE [Bussmann et al., 2025] などの改良が提案されている。それでも、probing と steering の間にはギャップが残ると報告されている [Wu et al., 2025]。

動作生成モデルでも、内部表現へ介入して動作を変える研究が報告されている。π0 と OpenVLA の FFN の活性から意味のある方向を取り出し、動作の速さや方向を変えた研究 [Häon et al., 2025] や、VLA に SAE を適用し、動作プリミティブに対応する特徴を強めて動作を変えた研究 [Swann et al., 2026] がある。一方で、VLA の中には汎用的な動作プリミティブに対応する特徴と、特定のエピソードを丸暗記した特徴の両方があること [Swann et al., 2026]、LLM 向けの解釈手法はそのまま VLA には移らず、SAE は疎ではあるが不完全な介入の基底であること [Jin et al., 2026] も指摘されている。パラメータの少ない特化モデルは、汎用的な動作の要素よりもデモの動作そのものを丸暗記するように学習しやすいと考えられ、その場合 steering はさらに難しくなると予想した。

1.4 想定する場面とタスク

生産ラインで複数種類のパーツを仕分ける作業を想定する。パーツ A は A の箱へ、パーツ B は B の箱へ運ぶデモで学習したあとに、「やはりパーツ A は B の箱の位置へ、パーツ B は A の箱の位置へ運びたい」

という変更が必要になる場面である。

本課題では、パーツを4色（赤・緑・青・黄）のキューブに見立てた。机の上に色の違う4つの箱が固定位置にあり、4色のうち1色のキューブが1つ置かれている。Panda アームがキューブを掴み、キューブと同じ色の箱に入れるのがタスクである。方策には ACT（Action Chunking with Transformers）を使い、スクリプト方策で生成したデモから模倣学習した。これは、パラメータ数が少なく、動作データだけで学習した特化モデルにあたる。そのうえで、学習済みの重みは固定したまま、推論時に SAE または DiffMean で内部の活性へ介入し、「赤いキューブを緑・青・黄の箱に入れる」ことができるかを調べる。

1.5 目的

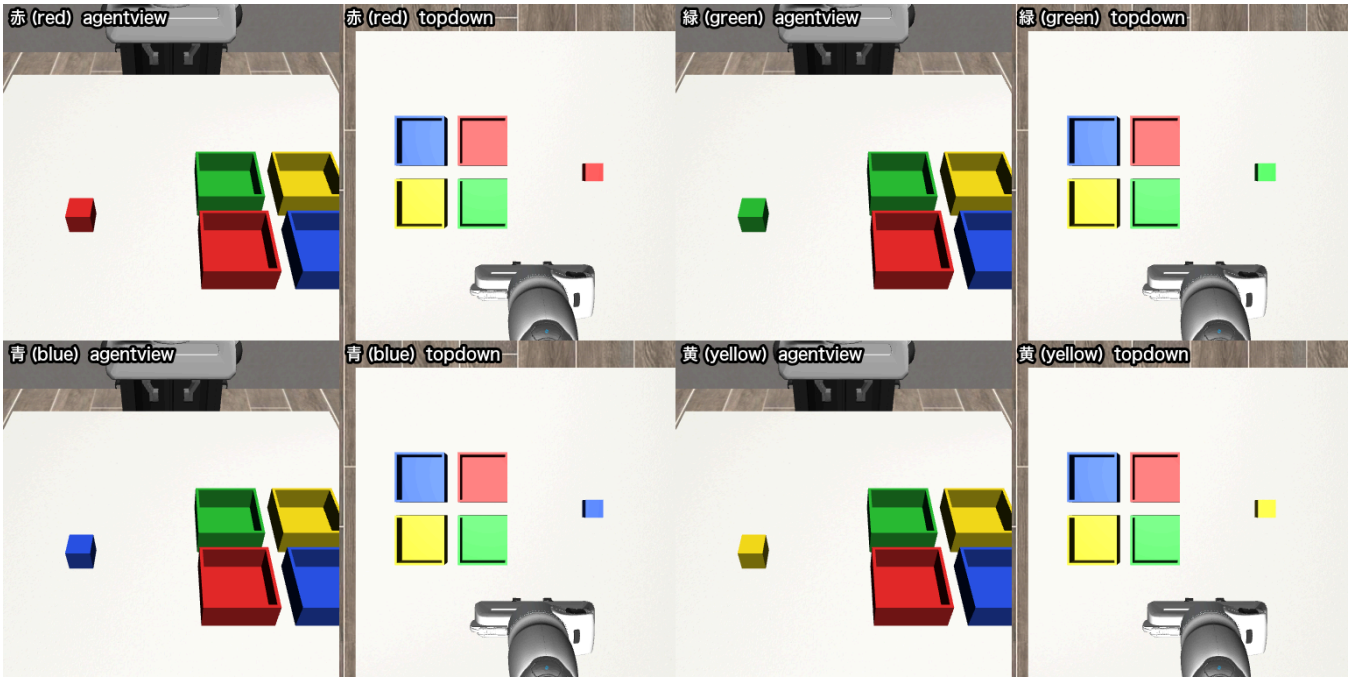
本課題の目的は、パラメータの少ない動作生成モデル（ACT）に SAE を適用する再現実験を行い、probing と steering のギャップがどの程度現れるかを検証することである。介入は3種類で、それぞれ SAE と DiffMean で比較する。1つ目は**色**の介入（意図レベル）で、キューブの色の表現を別の色に置き換えて運び先の箱を変える。2つ目は **x/y 移動**の介入（運動レベル）で、手先を x/y 方向へずらす特徴を強める。3つ目は**軌道**の介入（運動レベル）で、4つの箱それぞれへ向かう軌道に対応する特徴を強める。これにより、「どの箱へ運ぶか」という意図レベルの介入と、「手先をどちらへ動かすか」という運動レベルの介入とで、到達できる範囲と副作用がどう違うかを明らかにする。

2. 環境とデータ

2.1 環境

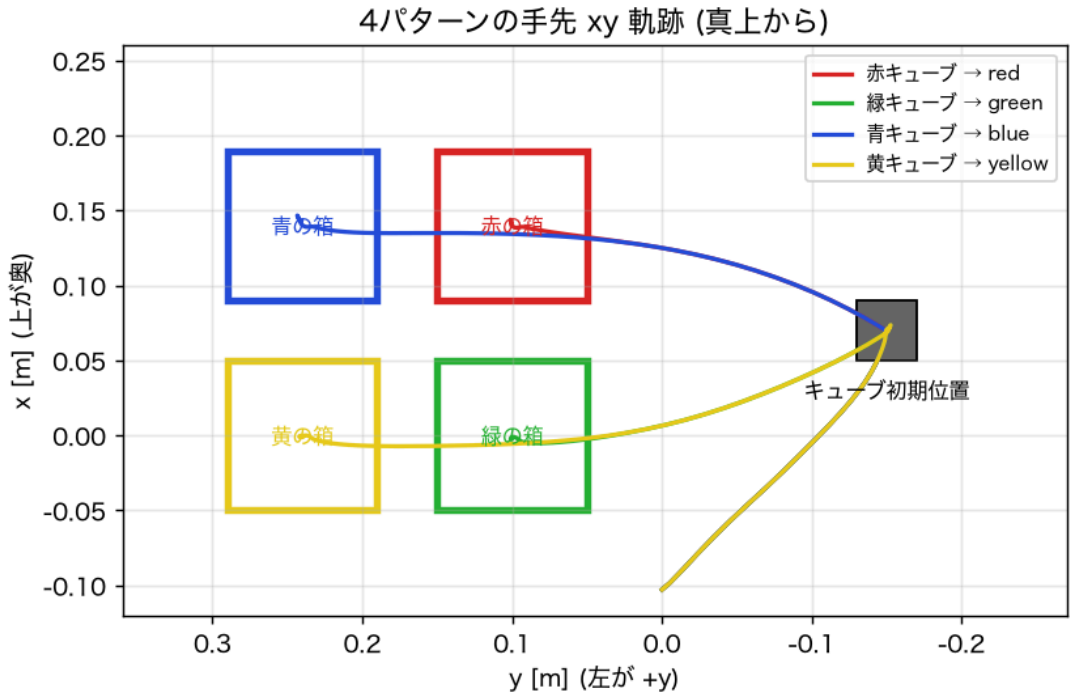
robosuite 1.5.1（MuJoCo 3.3）上に、`ManipulationEnv` を継承した自作環境 `CubeSortEnv` を作った。ロボットは Panda で、制御は OSC_POSE の差分指令、行動は位置3・回転3・グリッパー1の7次元、制御周期は 20Hz である。

配置はすべて固定で、キューブは右側の (x, y) = (0.07, −0.15) に置き、内寸 10cm の箱を左側に 2×2 で並べた（赤 (0.14, 0.10)、緑 (0.00, 0.10)、青 (0.14, 0.24)、黄 (0.00, 0.24)）。赤の箱から見ると、緑は x 方向だけ、青は y 方向だけずれた隣にあり、黄は x と y の両方がずれた対角にある。初期状態には、キューブの xy に ±1cm、向きに ±10°、関節角に ±0.02rad のノイズを加えた。配置はサンプルを確認しながら4回修正した。Panda の可動範囲に合わせ、キューブを右・箱を左に置き、不要な手首の回転をなくし、キューブの x を箱の2列の中間にした。



2.2 データ

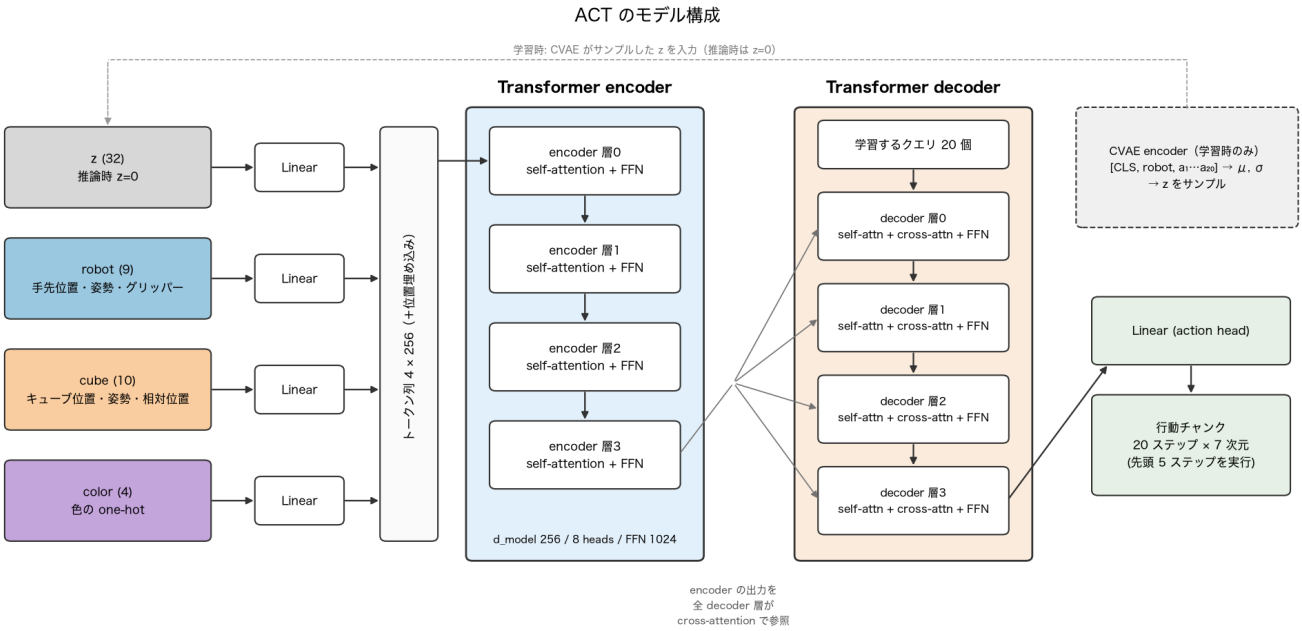
デモは、ウェイポイント（接近 → 降下 → 把持 → 持ち上げ → 運搬 → 下降 → 離す → 退避）を手先位置の P 制御で追うスクリプト方策で生成し、新たな人手デモは集めていない。色ごとに 100本、合計 400 本で、成功条件を満たした割合（採用率）は4色とも 100% だった。デモに多様性を持たせるため、ウェイポイントの xy に ±1cm のノイズ、P ゲインに 0.8〜1.2 倍のばらつき、行動にガウスノイズ（σ=0.05）を加えた。



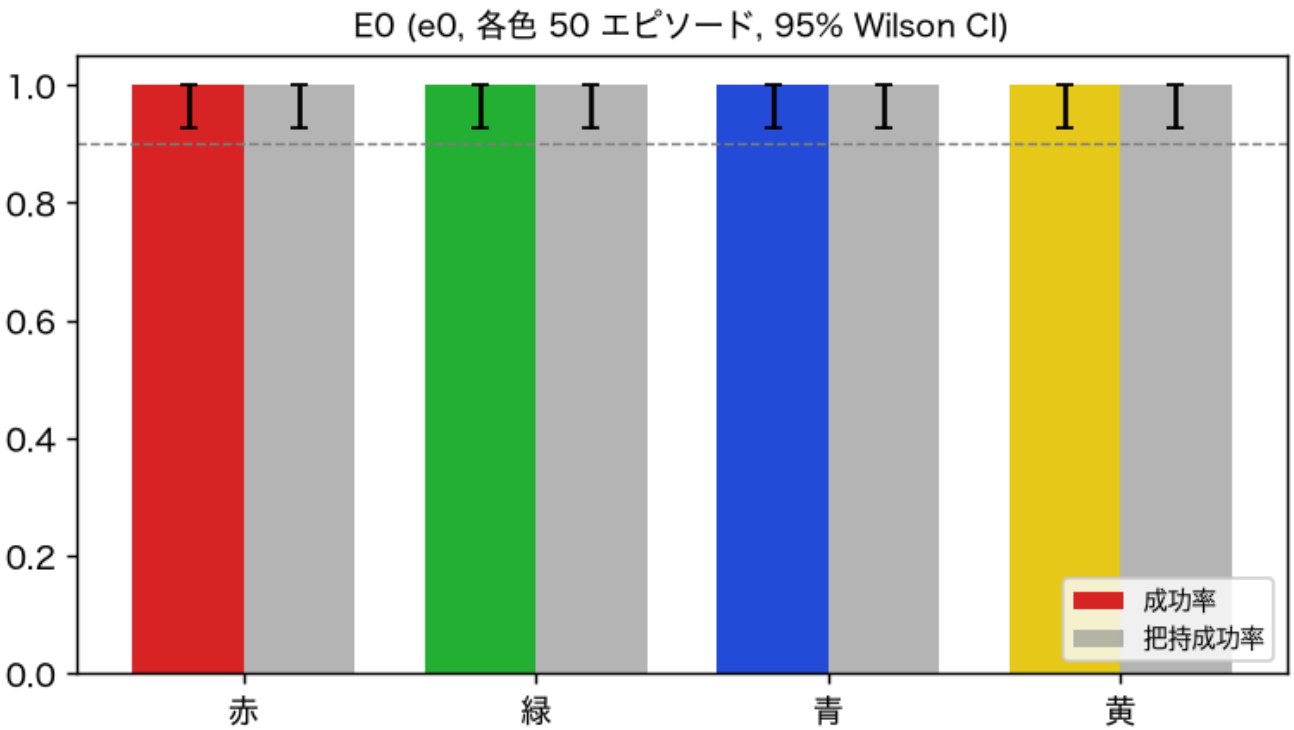
3. 手法

3.1 ACT

最小構成の ACT を自前で実装した。入力は3つのトークンで、robot トークンは手先の位置・姿勢・グリッパーの開き（9次元）、cube トークンはキューブの位置・姿勢・手先からの相対位置（10次元）、color トークンはキューブの色の one-hot（4次元）である。チャンク長は 20 で、1回の推論で実行するのは 5 ステップとした。d_model は 256、encoder と decoder はそれぞれ 4層、CVAE の潜在は 32次元で、推論時は z=0 とした。Temporal Ensembling は使っていない。過去のチャンクを混ぜると介入の効果がなまり、元の動作へ戻る力が強まるためである。推論時に使うパラメータ（CVAE encoder を除く）は約 740 万、学習時に使う CVAE encoder を含めると約 1,060 万である。モデルの構成を図に示す。学習は 30k step で、MPS で約25分かかった。介入なしのベースライン（E0）では、4色とも成功率 100%（各 50エピソード、95% 信頼区間 [93%, 100%]）だった。



実験はすべて1台のノート PC で行った。マシンは MacBook Air（Apple M5、10 コア CPU（高性能 4 + 高効率 6）、メモリ 32GB、macOS 26.5）で、学習と推論には PyTorch 2.15 の MPS バックエンド（Apple GPU）を、シミュレーションには MuJoCo の CPU 実行を使った。外部の GPU サーバーは使っていない。

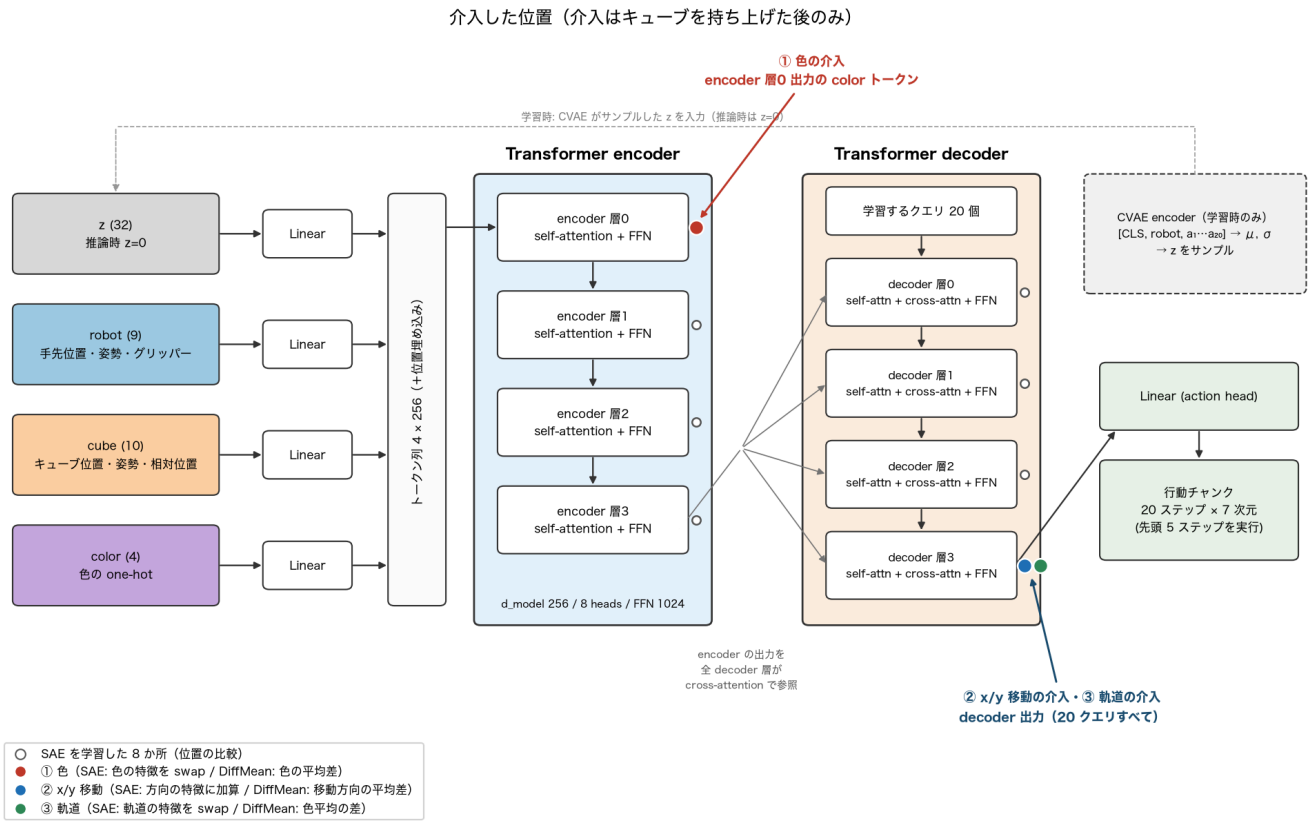


3.2 SAE と介入

encoder 各層の出力（4か所）と decoder 各層の出力（4か所）に TopK SAE（辞書 2048、k=32/16）を学習し、加えて color トークンだけで学習した「色の SAE」も作った。介入は 1.2 節で述べた誤差を保存する形 $\mathbf{h}' = \mathbf{h} + W_{\text{dec}}(\mathbf{f}' - \mathbf{f})$ にし、SAE が再構成できない成分は壊さないようにした。比

較に使う DiffMean は、平均活性の差ベクトルを足す（$\mathbf{h}' = \mathbf{h} + \alpha\mathbf{v}$）。介入は把持後（キューブを持ち上げた後）だけに入れ、そのタイミングの判定にはシミュレータの真の状態を使った。

調べた介入は次の3種類で、介入した位置を図に示す。

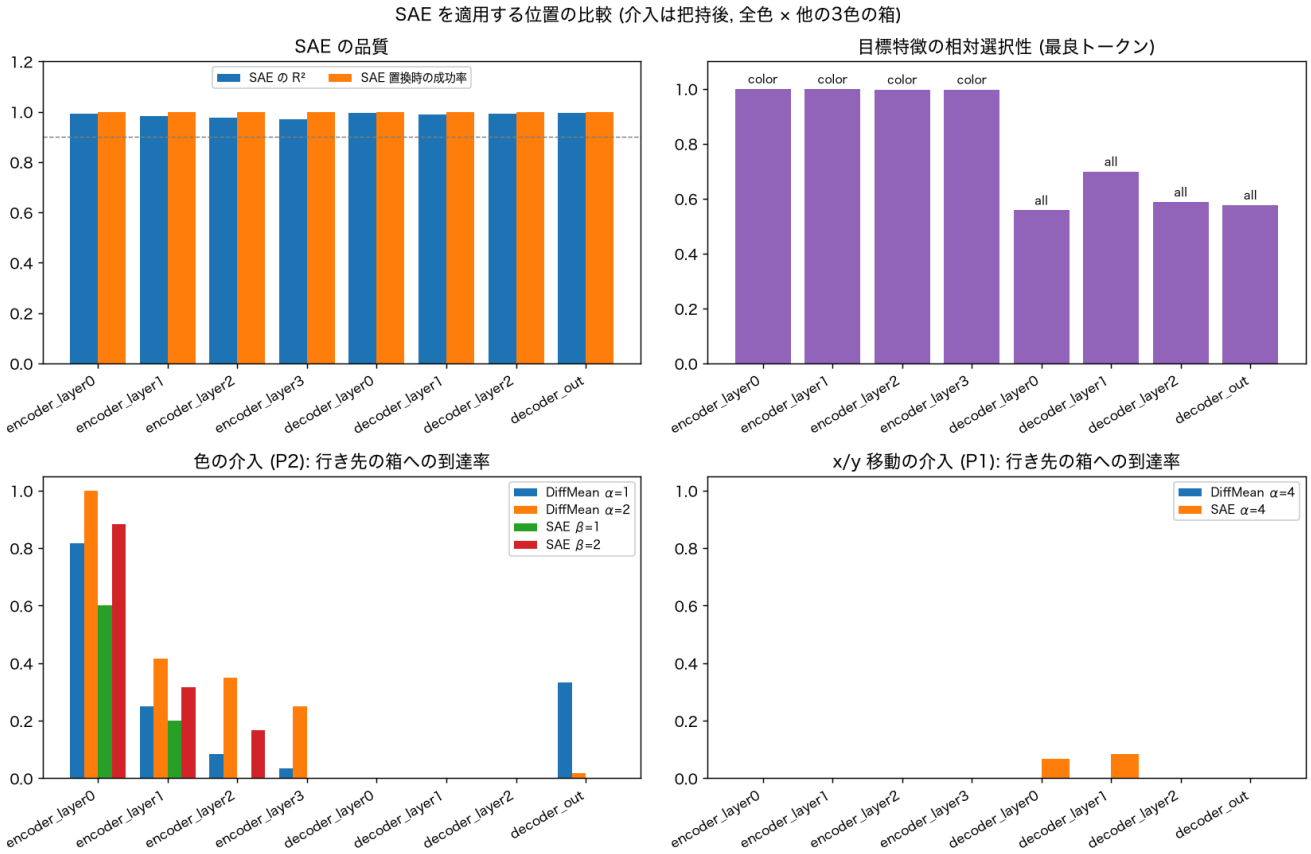


介入	位置	SAE	DiffMean
① 色（意図レベル）	encoder の color トークン	元の色の特徴を 0 にし、行き先の色の特徴をその色での平均 × β にする	μ_行き先 − μ_元 を α 倍して足す
② x/y 移動（運動レベル）	decoder 出力	+x/−x/+y/−y の方向の特徴に α を足す	手先が ±x / ±y に動くときの平均活性の差を足す
③ 軌道	decoder 出力	色ごと（＝軌道パターンごと）の特徴を swap する	位置 k ごとの色平均の差を足す

4. 結果

4.1 SAE の品質と、介入する位置の比較

SAE の再構成は 8か所すべてで $R^2 \geq 0.97$ で、SAE で置き換えても成功率は 100% のままだった。ただし、encoder の浅い層では死んだ特徴が 10% を超えた（層0 で 30%）。色の介入は encoder の最初の層（color トークン）が最もよく効き、深い層ほど効かなくなった。

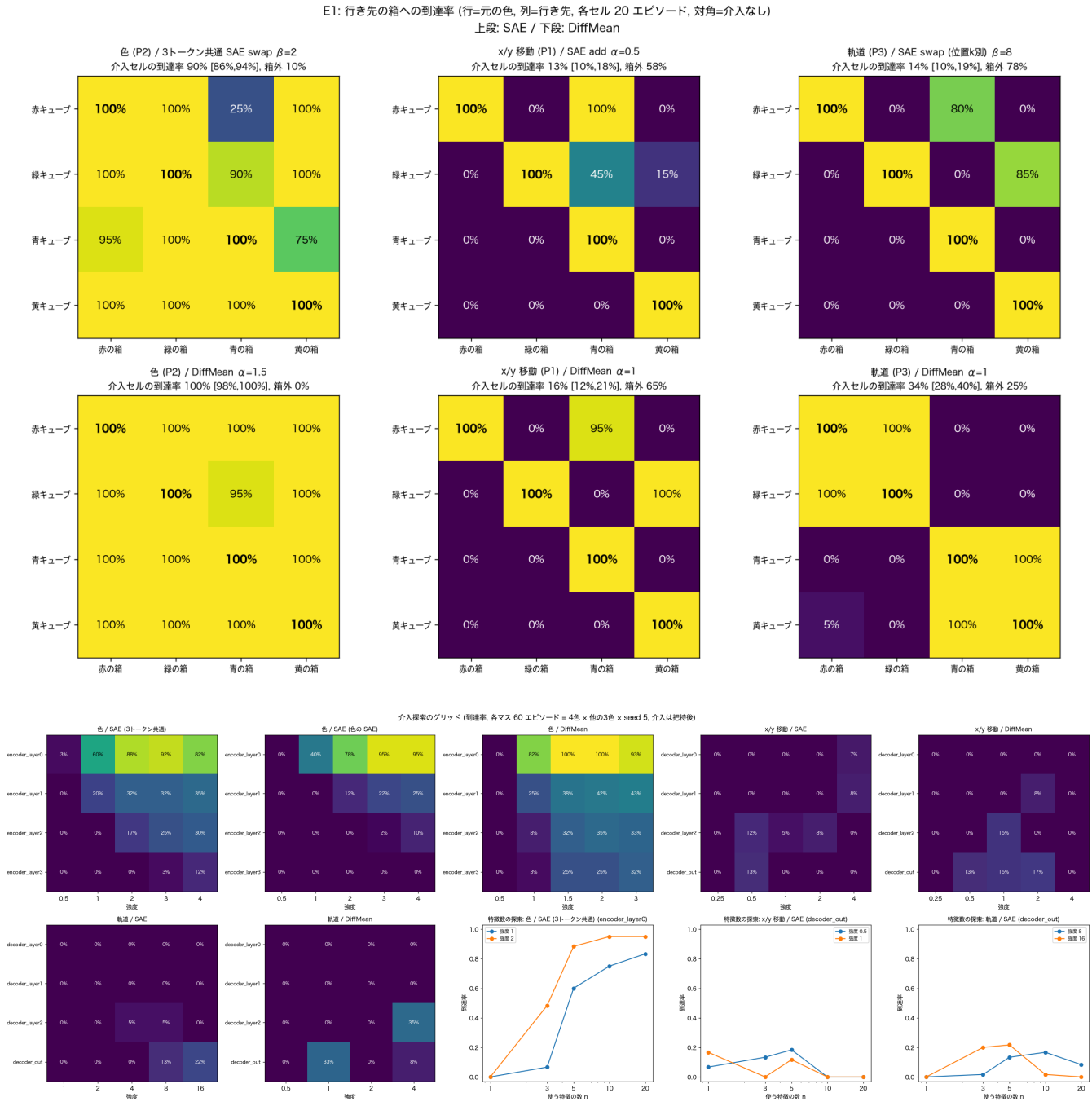


4.2 E1: 介入3種類 × SAE / DiffMean

行き先の箱への到達率（12通り × 20エピソード）を下表に示す。

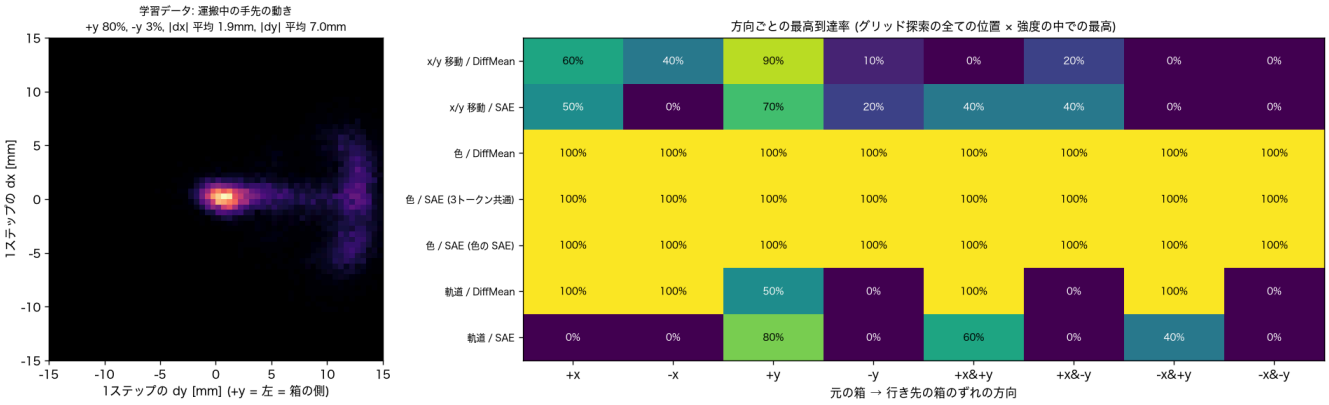
介入	SAE	DiffMean
① 色	90%（3トークン共通の SAE）/ 94%（色の SAE）	**100%**
② x/y 移動	13%	16%
③ 軌道	14%	34%

色の介入は、エピソードの最初から入れても把持は崩れず、行き先だけが変わった。一方、x/y と軌道の介入はほとんど効かなかった。探索が不十分だった可能性を確かめるため、位置（候補の全層）× 強度（5段階）× 特徴数（1〜20）の 166通り（9960エピソード）を試したが、x/y は最高 17%、軌道は最高 35% にとどまった。

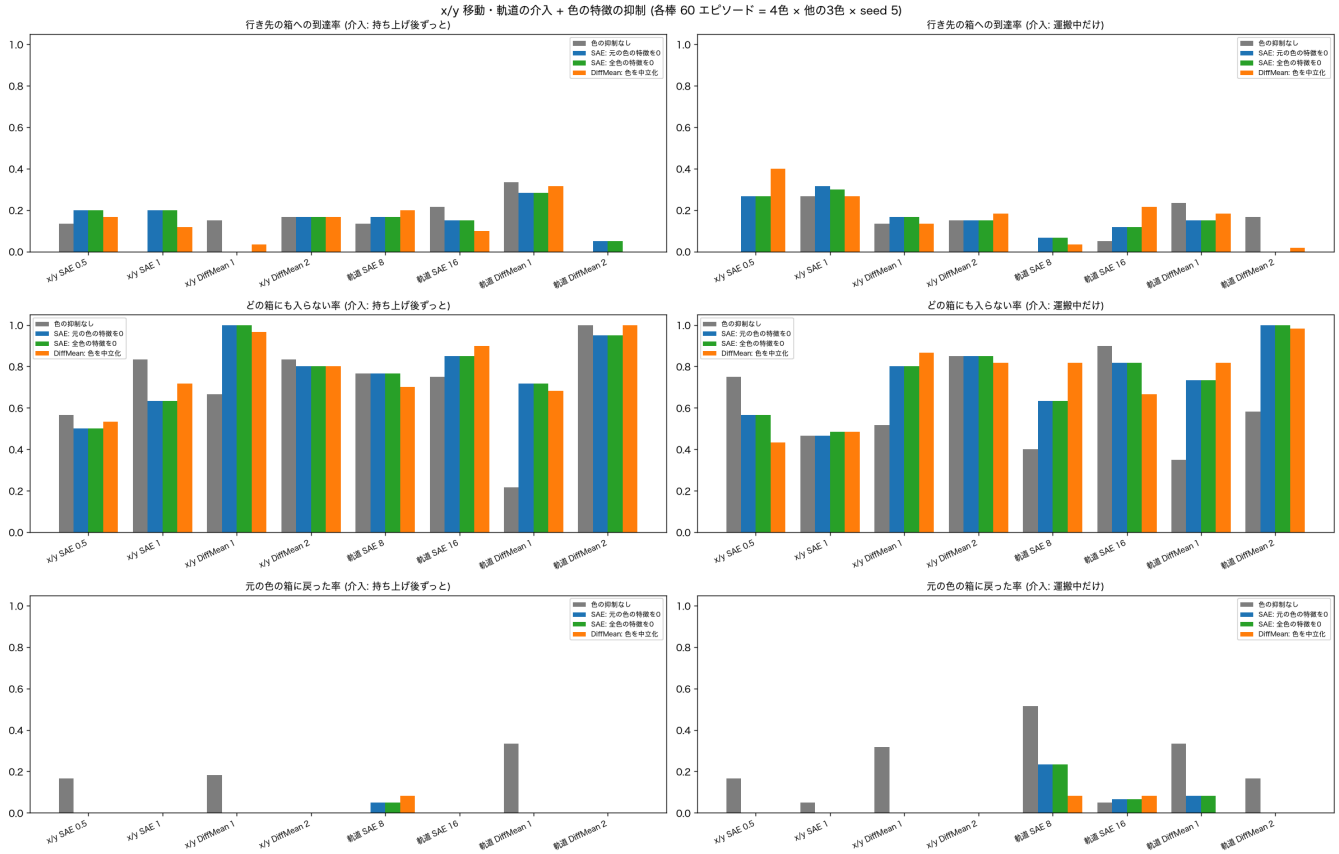


4.3 x/y と軌道が効かなかった要因

まず、データに偏りがあった。運搬中の手先の動きは +y が 80% を占め、−y は 3% しかない。x/y と軌道の介入が成功したのも、ほぼ +y 方向だけだった。

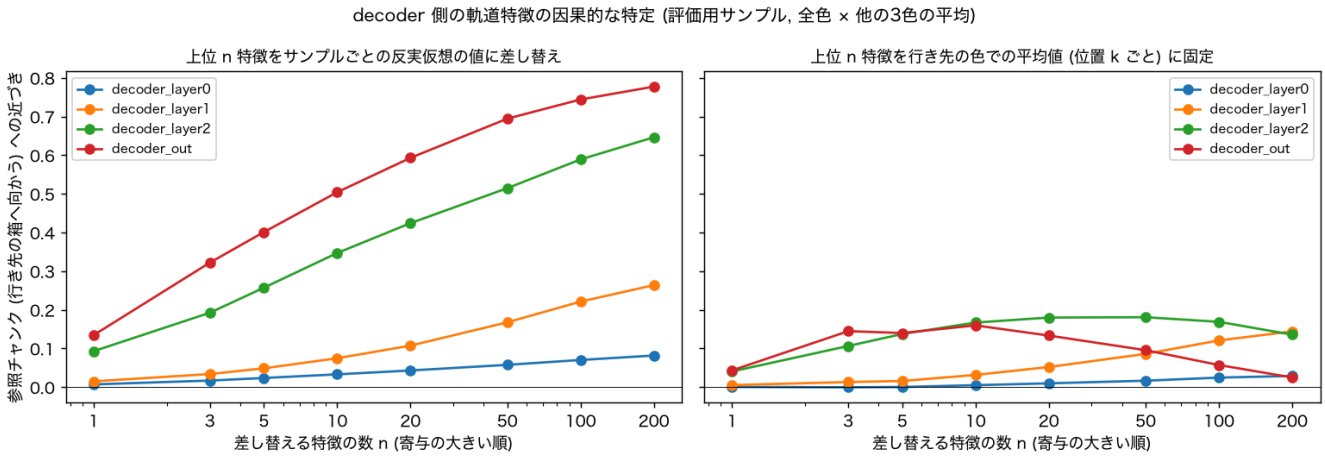


次に、元の箱への引き戻しを調べた。色の特徴を同時に抑えると、元の箱へ戻る率は 14% から 2〜3% に下がった。しかし到達率は上がらず、代わりに箱の外に置く割合が増えた。色を抑えるだけでは、キューブはどの箱にも置かれなかった。

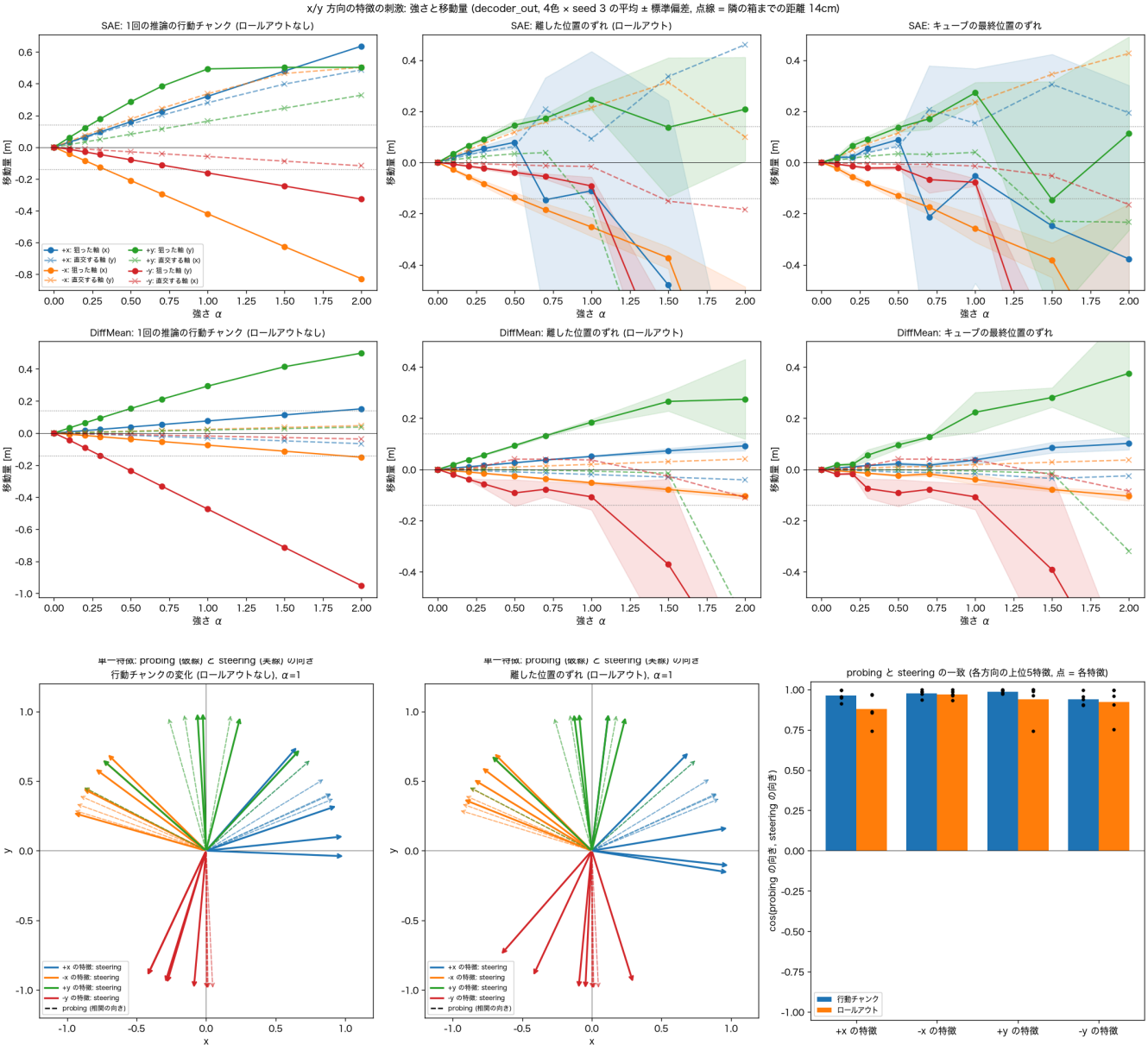


さらに、行動チャンクを直接比較した。color トークンを行き先の色に変えたときに方策が出すチャンクを基準とし、各介入でチャンクがそこへどれだけ近づくかを測ると、色の介入は 81〜89%、x/y は 20〜33%、軌道の SAE は −1% だった。x/y の介入は向きは合っているものの、状態によらない一定の押しにしかなっていなかった。

軌道の特徴については、行き先の情報は decoder の活性から線形に約 100% 読み出せるのに、色ごとに選んだ 20特徴では 59〜73% しか読めなかった。反実仮想（color だけを変えた実行）との差と勾配で特徴を選び直すと、decoder 出力の約 50特徴（2048 中）を状態に応じた値に差し替えれば軌道が 90% 切り替わったが、決まった値に固定すると 2〜8% にとどまった。また、4パターンの違いは、関節角度でも内部表現でも「x の行き先 + y の行き先」の足し算でほぼ説明でき、パターン固有の部分は関節角度で 0.6%、decoder で 4〜13% しかなかった。



最後に、probing と steering の一致を調べた。1つの特徴について、データ上の相関の向きと、刺激したときの動きはよく一致した（cos 0.88〜0.99）。しかし、1個で x 軸に沿って動く特徴はなかった。

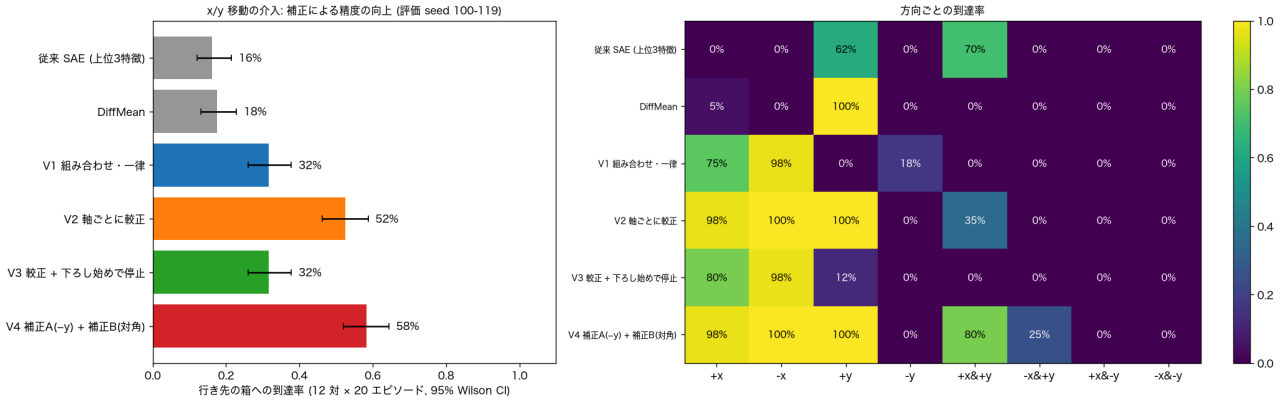


4.4 x/y 移動の介入の改善

x/y の介入を次の手順で改善した。まず、特徴を組み合わせて選び、7次元すべての行動成分について狙った軸以外が 0 になる組み合わせを NNLS で求めた。次に、軸ごとに強さを較正した。さらに、対角の方向は対ごとに強さを決め、–y を含む方向では下ろし始めたら介入を止めるようにした。評価には、調整に使っていない seed を使った。

条件	到達率
従来の SAE（上位3特徴）	16%
DiffMean	18%
組み合わせで選ぶ	32%
+ 軸ごとに較正	52%
+ 対角の補正・下ろし始めで停止（–y を含む方向のみ）	**58%**
上に色の抑制を追加	36〜48%

改善の結果、1軸の移動は +x 98%、–x 100%、+y 100% まで届くようになった。一方、–y を含む方向はどの補正でも 0% だった。方策は運搬の高さで止まったまま、上限の 400ステップに達して終わった。



5. 考察

行き先を自在に変えられたのは、目的（色）の表現への介入だけだった。 color トークンを差し替えると、cross-attention を通じて decoder のすべての層の計算が変わり、今の状態に合った「行き先の箱への計画」がそのまま出てくる。

運動レベル（x/y）の介入は、目的が決める「止まる位置」をずらす形でしか働かない。 方策は、自分の目的（元の箱）に着いたと判断したときに下ろし始める。そのため、元の箱を通り過ぎる方向や同じ距離の方向へは、補正すれば届く。しかし、目的より手前で置く必要がある方向（–y）には届かない。色を抑えても、届く方向と届かない方向が入れ替わるだけだった。

decoder 側に「4パターンの軌道」の記号はない。 軌道は x と y の成分に分かれ、状態に応じて値が変わる形で表現されている。そのため、決まった値で刺激する介入では軌道全体を切り替えられない。これは、特徴を見つけられても（probing）それを刺激して動作を変えられない（steering）というギャップが、小さな動作生成モデルでもはっきり現れることを示している。

SAE と DiffMean を比べると、色の介入では DiffMean（100%）が SAE（90〜94%）を上回った。一方、x/y の介入では、SAE の特徴を組み合わせて使うことで DiffMean（18%）を大きく上回った

（58%）。SAE の利点は、行動の成分ごとに分解し、狙った方向だけに効く組み合わせを作れる点にあった。

本研究には限界もある。色と行き先は常に1対1で、箱の配置も固定（方策は箱を観測していない）なので、色の特徴が「色の識別」なのか「箱 c 行き」なのかはデータから区別できない。また、介入するタイミングの判定にシミュレータの真の状態を使っている。結論は、2×2 の4パターン、Temporal Ensembling なし、辞書 2048 の TopK SAE という条件でのものである。

6. 工夫した点、うまくいかなかった点と反省

工夫した点

「色 / x/y / 軌道」の3種類の介入を SAE と DiffMean で同じ条件で比べ、位置・強度・特徴数を網羅的に探索して到達率の上限を示した。効かなかった理由は、データの偏り、行動チャンクの直接比較、反実仮想による因果的な特徴選択、関節角度の分解によって切り分けた。そのうえで x/y 介入の失敗を、特徴の組み合わせ選択（グリッパーなど他の成分の混入の除去を含む）と較正によって 16% から 58% に改善し、残る失敗（–y）の理由も示した。また、調整と評価で seed を分け、結果を合わせ込まないようにした。

うまくいかなかった点と反省

最初は別の仕様書（MimicGen データ版）で作業を始めてしまい、時間を失った。また、x/y の特徴を「上位3つを独立に選ぶ」方法で選んでいたため、斜めの特徴が足し合わさり、最初は運動レベルの介入を過小評価していた。特徴の組み合わせを選ぶときも、最初は位置しか条件にしておらず、グリッパーの成分が混ざってキューブを落としてしまった。

–y（目的より手前で置く方向）は、運動レベルの介入だけでは解決できなかった。方策の「着いたら下ろす」という判断に介入する方法（目的の表現と組み合わせるなど）は今後の課題である。また、色と行き先を切り分けるには、箱の配置を入れ替えたり画像入力にしたりする設定が必要だった。

引用

- Y. Zhu et al., "robosuite: A Modular Simulation Framework and Benchmark for Robot Learning," arXiv:2009.12293.
- T. Z. Zhao, V. Kumar, S. Levine, C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," RSS 2023.
- E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," IROS 2012.
- L. Gao et al., "Scaling and evaluating sparse autoencoders," arXiv:2406.04093.
- A. Templeton et al., "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet," Transformer Circuits Thread, 2024.
- S. Marks, M. Tegmark, "The Geometry of Truth: Emergent Linear Structure in Large Language Model Representations of True/False Datasets," arXiv:2310.06824.
- Z. Wu et al., "AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders," arXiv:2501.17148.

- B. Bussmann et al., "Learning Multi-Level Features with Matryoshka Sparse Autoencoders," arXiv:2503.17547.
- B. Häon et al., "Mechanistic interpretability for steering vision-language-action models," CoRL 2025, arXiv:2509.00328.
- A. Swann et al., "Sparse Autoencoders Reveal Interpretable and Steerable Features in VLA Models," arXiv:2603.19183.
- X. Jin et al., "Event-Grounded Sparse Autoencoders for Vision-Language-Action Policies," arXiv:2605.17204.